

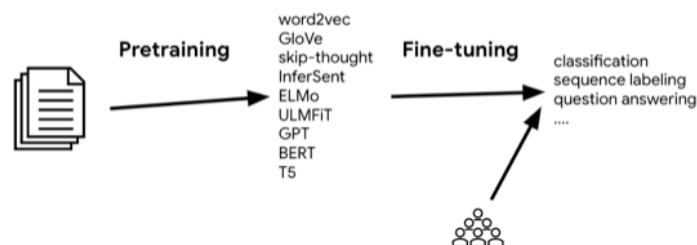
Parameter-efficient Finetuning

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

Full fine-tuning becomes too expensive at LLM scale → ICL avoids training but pays prompt cost and instability → PEFT freezes pretrained parameters θ and trains small module parameters ϕ → Three PEFT families: parameter composition, input composition, function composition → Parameter composition changes selected weights or weight updates → Low-rank assumption motivates LoRA as trainable BA update → Input composition learns continuous prompts instead of hand-written prompts → Multi-layer prompt tuning injects prompts into attention keys and values → Function composition inserts adapters as bottleneck task modules → Practical comparison: LoRA is mergeable, prompts are tiny but capacity-limited, adapters are robust but add inference modules

Finetuning

- With increasing model size, fine-tuning becomes increasingly expensive
- The standard transfer learning formula breaks down



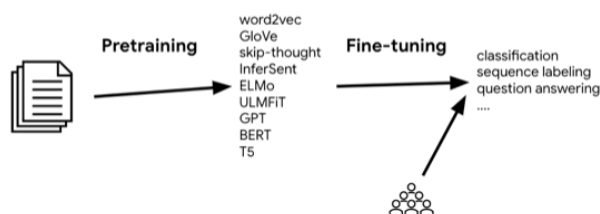
这节课一句话

这节课一句话：当 LLM 大到不能为每个任务 full fine-tune 时，PEFT 的思路就是冻结大模型，把任务适配信息塞进少量可训练参数里；三条路分别是改权重、改输入、加模块，其中 LoRA、prompt tuning 和 adapters 是最典型考点。

1. 为什么需要 PEFT: full fine-tuning 在 LLM 时代太贵

Finetuning

- With increasing model size, fine-tuning becomes increasingly expensive
- The standard transfer learning formula breaks down



人话直觉

想象你有一座巨型图书馆模型。过去为了服务一个新任务，我们可以把整座图书馆的书架都重新摆一遍；但 LLM 规模上来后，每个任务都重摆全馆，显存、训练时间、存储和部署都会爆炸。PEFT 的出发点不是“全量微调不好”，而是“全量微调太奢侈”。

精确定义 / 机制： Slide 2 点出主矛盾：with increasing model size, fine-tuning becomes increasingly expensive，因此 standard transfer learning formula breaks down。Full fine-tuning 要更新所有参数，还要在训练中保存 gradients、activations 和 optimizer states；这些开销通常比单纯 inference 更重。PEFT 的核心目标是冻结大部分 pretrained parameters θ ，只训练少量新增或被选中的 module parameters ϕ ，让多个任务共享同一个 base model，只切换小型任务适配器。这样牺牲一点自由度，换来显存、存储、多任务部署和维护成本的大幅下降。

考试问法：如果考“为什么需要 PEFT”，先说 full fine-tuning 更新全部参数，在 LLM 上训练显存、计算和存储成本过高；再说训练比推理更吃内存，因为需要梯度、激活和优化器状态；最后补 PEFT 冻结 base model、只训练少量参数，目标是接近 full fine-tuning 性能。

2. ICL 是绕路方案，但不是 PEFT 的替代品

Disadvantages of ICL

- **Inefficiency**: the prompt needs to be processed every time the model makes a prediction
- **Performance**: prompting generally performs worse than fine-tuning (Brown et al., 2020)
- **Sensitivity** to the wording of the prompt (Webson and Pavlick, 2022), order of examples (Zhao et al., 2021; Lu et al., 2022)
- **Lack of clarity** regarding what the model learns from the prompt — even random label can provide non-trivial results (Min et al., 2022)!

人话直觉

In-context learning 像你不培训员工，只是在每次任务开始前塞一大段说明书和例题。好处是不用训练；坏处是每次都要读说明书，而且说明书的措辞、例题顺序、甚至随机标签都可能改变行为。PEFT 则像给员工装一个小插件，训练一次后任务习惯被存在参数里。

精确定义 / 机制：Slide 4 列出 ICL 的四个关键问题。第一是 inefficiency: prompt 和 demonstrations 每次预测都要重新处理，占用上下文窗口并增加推理成本。第二是 performance: prompting 通常不如 fine-tuning 稳定或强。第三是 sensitivity: 模型对 prompt wording 和 examples order 很敏感。第四是 lack of clarity: random labels 也能带来 non-trivial results，说明模型到底从 prompt 学到什么并不透明。PEFT 与 ICL 都是在避免 full fine-tuning，但区别是 ICL 不更新参数、把任务信息临时放进输入；PEFT 需要训练，但把任务适配信息压进少量可训练参数。

考试问法：如果问 PEFT vs ICL，答：ICL 不训练参数、靠 prompt 临时适配，优点是方便；缺点是每次推理都处理 prompt、性能和 prompt wording/order 敏感。PEFT 训练少量参数，成本高于纯 prompting，但通常更稳定，并且不需要每次携带长示例。

3. 三类 PEFT: 到底是在改权重、改输入, 还是加函数

Three Kinds of PEFT

Let $f: X \rightarrow Y$ be a neural network, which can be decomposed into a composition of functions $f_{\theta_1} \circ \dots \circ f_{\theta_n}$, where each function has parameters θ_i with $i \in \{1, \dots, n\}$. A module with parameters ϕ can modify a function f_{θ_i} as follows:

- Parameter composition: $g_i(x) = f_{\theta_i \oplus \phi}(x)$ (interpolation)
- Input composition: $g_i(x) = f_{\theta_i}([x, \phi])$ (concatenation)
- Function composition: $g_i(x) = f_{\theta_i} \circ f_{\phi}(x)$

Typically, only module parameters ϕ are updated while θ is fixed

人话直觉

把 Transformer 想成一条流水线。PEFT 可以从三个位置下手: 第一, 调机器里的某些旋钮, 也就是 parameter composition; 第二, 给流水线入口多放一张可学习说明卡, 也就是 input composition; 第三, 在流水线中间插一个小加工站, 也就是 function composition。

精确定义 / 机制: Slide 10 给出统一 notation: 神经网络 $f: X \rightarrow Y$ 可以分解为 $f_{\theta_1} \circ \dots \circ f_{\theta_n}$, 每个子函数有原始参数 θ_i 。PEFT 引入 module parameters ϕ , 但 typically only ϕ is updated while θ is fixed。Parameter composition 写作 $g_i(x) = f_{\theta_i \oplus \phi}(x)$, 意思是通过稀疏、结构化或低秩方式修改参数; input composition 写作 $g_i(x) = f_{\theta_i}([x, \phi])$, 意思是把可学习向量拼到输入侧; function composition 写作 $g_i(x) = f_{\theta_i} \circ f_{\phi}(x)$, 意思是新增一个任务特定函数模块。

$$f = f_{\theta_1} \circ \dots \circ f_{\theta_n}, \quad g_i(x) = f_{\theta_i \oplus \phi}(x), \quad g_i(x) = f_{\theta_i}([x, \phi]), \quad g_i(x) = f_{\theta_i} \circ f_{\phi}(x)$$

考试问法: 三类 PEFT 是本讲最稳的综合题。不要只列名字, 要用一句话区分: parameter composition 改权重或权重更新, 例 LoRA/BitFit; input composition 改输入, 例 prompt tuning; function composition 加新模块, 例 adapters。

4. Parameter composition: 少动、按组动、低秩动

Parameter composition

- **Sparse Subnetworks**, where module parameters ϕ are enforced to be sparse
- **Structured Composition**, where we impose a structure on the weights θ_i that we select - e.g., we update the weights belonging to a pre-defined group
- **Low-Rank Composition**, where the module parameters ϕ lie in a low-dimensional space

人话直觉

如果必须改一台巨大机器，不一定要拆掉全部零件。你可以只剪掉或打开少数线路；也可以规定只调某一组旋钮，比如 bias；还可以相信真正有用的调整其实只有几个方向，于是用低维坐标描述它。

精确定义 / 机制：Slide 12 把 parameter composition 分成三类。Sparse subnetworks 强调 sparsity，常见思路是 pruning 或 mask，只让一部分连接或参数参与适配。Structured composition 预先定义可训练集合 $\mathcal{G}$ ，例如某层、某组层、某类组件；BitFit 就是只更新 bias vectors 的例子。Low-rank composition 假设任务所需更新不需要完整高维空间，而是位于低维子空间。它们的共同点是都在参数侧做文章，但归纳偏置不同：稀疏性强调“少数连接够用”，结构化强调“某些组件最值得动”，低秩强调“有效更新维度很低”。

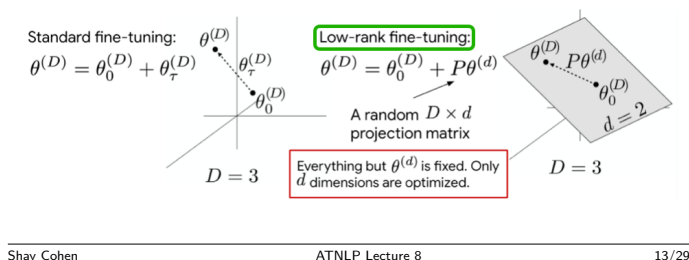
考试问法：如果问 parameter composition 的子类，答 sparse subnetworks、structured composition、low-rank composition；每类给一句机制和一个例子。BitFit 属于 structured composition；LoRA 属于 low-rank/parameter composition。

5. 低秩动机：任务更新可能只需要低维方向

Parameter composition: low-rank composition

Another useful inductive bias: module parameters ϕ should lie in a low-dimensional space. Li et al., 2018 show that models can be optimised in a low-dimensional, randomly oriented subspace rather than the full parameter space.

Low-rank finetuning takes the form $g = f_{\theta+P\phi}$ where $P \in \mathbb{R}^{D \times d}$ - with a dense matrix of shape $D \times d$, this scales as $O(D \times d)$ in time and storage. D is the dimension of the model parameter, and d is a reduced size.



Shay Cohen

ATNLP Lecture 8

13/29

原 PPT 第 15 页

人话直觉

LLM 参数空间像一个超高维宇宙，但具体任务可能只需要沿少数几条方向移动。低秩方法的赌注是：别在整个宇宙里学每个坐标，先在低维小房间里学 ϕ ，再用矩阵 P 把它投影回原参数空间。

精确定义 / 机制： Slide 15 说明 low-rank composition 的基础：module parameters ϕ lie in a low-dimensional space。形式上，低秩 fine-tuning 可写为 $g = f_{\theta+P\phi}$ ，其中完整模型参数维度是 (D) ，低维参数维度是 (d) ，映射矩阵 $P \in \mathbb{R}^{D \times d}$ 。这样做把可训练自由度从完整 (D) 维压到 (d) 维；但如果 P 是 dense matrix，本身也可能带来 $O(Dd)$ 的时间和存储问题。LoRA 的进步点就在于不用固定随机投影，而是直接学习一个低秩权重增量的分解。

$$g = f_{\theta+P\phi}, \quad P \in \mathbb{R}^{D \times d}, \quad d \ll D$$

考试问法：考 low-rank composition 时，关键不是背“低秩省参数”，而是说明假设：有效任务更新位于低维子空间。解释 (D) 是完整参数维度， (d) 是缩小后的维度；再自然引到 LoRA。

6. LoRA: 冻结原权重，用 BA 学一个低秩增量

Parameter composition: LoRA

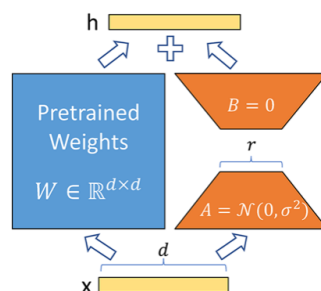
Low-Rank Adaptation — instead of learning a low-rank factorisation via a random matrix P , we can learn the projection matrix directly. LoRA (Hu et al., 2022) learns two matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ that are applied to the self-attention weights:

$$h = [W_0 + \Delta W]x = [W_0 + BA]x$$

In our notation:

$$g_i = f_{\theta_i + B_i A_i}, \forall f_i \in G$$

Food for thought: Why initialise that way? Keep the initial point the same, but make sure gradient evolves



Shay Cohen

ATNLP Lecture 8

14/29

原 PPT 第 17 页

人话直觉

LoRA 最好想成“不要重写原书，只贴一张可训练修订贴”。原权重 (W_0) 是冻结的旧书内容； (BA) 是小而灵活的修订贴。贴纸的面积远小于整页，但如果贴在关键位置，比如 attention 投影矩阵上，就能显著改变模型在任务上的行为。

精确定义 / 机制： Slide 17 给出核心公式： $(h=[W_0+\Delta W]x=[W_0+BA]x)$ 。LoRA 学两个矩阵 $(B \in \mathbb{R}^{d \times r})$ 与 $(A \in \mathbb{R}^{r \times k})$ ，让它们的乘积 (BA) 表示权重更新 (ΔW) 。因为 $\text{rank}(r)$ 通常远小于原矩阵维度，所以训练参数量远小于直接训练 (ΔW) 。讲义 notation 中，对选定函数集合 (G) 应用 $(g_i=f_{\theta_i+B_i A_i})$ 。初始化问题也很重要：常让初始 $(BA=0)$ ，使训练一开始模型行为等于原预训练模型，避免破坏已有能力；同时保留梯度路径，让 (A,B) 后续能学出更新。LoRA 常用于 self-attention 的 (W_Q, W_K, W_V) 或其他投影矩阵，并且推理时可把 (BA) 合并进原权重，推理路径很高效。

$$h = (W_0 + \Delta W)x = (W_0 + BA)x, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

考试问法： 如果考 LoRA，标准答案要包含：冻结 (W_0) ，训练低秩矩阵 (A,B) ，用 (BA) 近似 (ΔW) ， $\text{rank}(r)$ 控制容量和参数量，常用于 Transformer attention 权重；初始化让 $(BA=0)$ 保持初始输出不变；推理可合并权重。

7. Input composition: 把 prompt 从人写文本变成可学习向量

Input composition

Augment the input of the model with a learnable vector ϕ :

$$g_i(x) = f_{\theta_i}([\phi_i, x])$$

Input Composition and Prompting — standard prompting can be seen as finding a discrete text prompt that, when embedded using the model's embedding layer, yields ϕ_i

However, models tend to be sensitive to the choice of the prompt (Webson and Pavlick, 2022) and the order of examples (Zhao et al., 2021; Lu et al., 2022)

人话直觉

手写 prompt 像用自然语言给模型递纸条，但模型不一定按人类直觉理解纸条。Input composition 更直接：不必让纸条能被人读懂，只要学出一串 embedding，让模型内部觉得“现在该按这个任务行动”。

精确定义 / 机制： Slide 21 的公式是 $g_i(x) = f_{\theta_i}([\phi_i, x])$ ：把 learnable vector ϕ_i 拼到输入前面。Standard prompting 可以看作 input composition 的离散版本：人写的 prompt 先经过 embedding layer，变成模型输入中的向量。Prompt tuning 则跳过人工找词，直接学习一段 continuous prompt embeddings。它的优点是参数极少，训练和存储都轻；缺点是适配容量有限，因为任务信息主要从输入侧进入模型，而且会占用上下文长度、增加每次推理要处理的 prompt tokens。它也继承了 prompt 类方法的敏感性，只是把“找好文字”变成“优化好向量”。

$$g_i(x) = f_{\theta_i}([\phi_i, x])$$

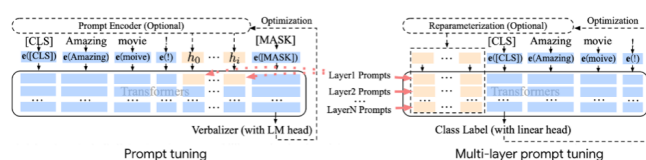
考试问法：如果问 prompt tuning，答：学习连续 prompt embeddings，并 prepend 到输入；它不同于 manual prompting，因为向量不一定对应可读文本；优点是参数量极小，缺点是容量有限、占上下文、在小模型和困难任务上可能表现较差。

8. Multi-layer prompt tuning: 不只在入口提示, 而是层层递纸条

Multi-layer prompt tuning

Instead of learning the module parameters ϕ_i only at the input layer, we can learn them at every layer of the model (Li and Jiang, 2021; Liu et al., 2022)

In practice, continuous prompts ϕ_i are concatenated with the keys and values in the self-attention layer (Li and Jiang, 2021)



人话直觉

普通 prompt tuning 只在模型入口放一排软 token, 像只在公司前台留便条。Multi-layer prompt tuning 认为这还不够, 于是每一层都给模型一点任务提示, 尤其把连续 prompts 拼到 self-attention 的 keys 和 values 里, 让各层都能“看见”任务偏好。

精确定义 / 机制: Slide 24 解释了为什么只在 input layer 学 ϕ_i 可能受限: 适配容量有限。进阶做法是在 every layer 学 module parameters, 实践中常把 continuous prompts ϕ_i concatenated with the keys and values in the self-attention layer。这样任务信息不只是作为开头 token 被第一层处理, 而是能在每层 attention 计算中直接影响模型可关注的 key/value memory。它比单层 prompt tuning 容量更强, 但代价是参数和计算更多, 也可能继续增加 attention 负担。考场上要把它和 LoRA 区分开: multi-layer prompt tuning 改的是 attention 的输入侧 K/V 序列; LoRA 改的是 attention 投影权重的低秩增量。

考试问法: 若问 multi-layer prompt tuning, 相比普通 prompt tuning 的优势是容量更高、影响多个层级; 机制是把 continuous prompts 加到每层, 常拼接到 self-attention keys/values; 代价是更多参数和计算。

9. Adapters: 在主模型里插一个瓶颈小模块

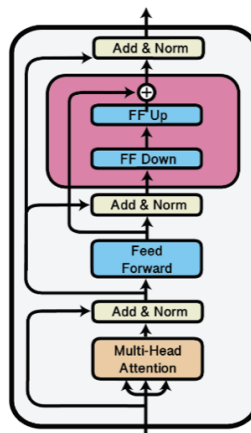
Adapters

The main purpose of functions f_{ϕ_i} added to a pre-trained model is to adapt it to a new task — these functions are also known as **adapters**

In NLP, an adapter in a Transformer layer typically consists of a feed-forward down-projection $W_D \in \mathbb{R}^{k \times d}$, a feed-forward up-projection $W_U \in \mathbb{R}^{d \times k}$, and an activation function σ (Houlsby et al., 2019)

$$f_{\phi_i}(x) = W_U [\sigma(W_D x)]$$

Adapter usually placed after multi-head attention and/or after the feed-forward layer



人话直觉

Adapter 像在高速公路旁边加一条短小匝道：主路 θ 不重修，车流经过时分出一点到小模块里压缩、变换、再并回去。这个小模块承载任务特定知识，所以每个任务只需要换匝道，不必换整条高速。

精确定义 / 机制：Slide 28 把 adapters 作为 function composition 的典型例子。Adapter 是加到 pretrained model 中的 task-specific function f_{ϕ_i} ，Transformer 中通常包含 down-projection $(W_D \in \mathbb{R}^{k \times d})$ 、activation (σ) 、up-projection $(W_U \in \mathbb{R}^{d \times k})$ ，公式为 $f_{\phi_i}(x) = W_U [\sigma(W_D x)]$ 。其中 (k) 是 bottleneck size，先降维限制参数数量和容量，再升维回 hidden size 以便和主模型表示结合。Adapters 通常放在 multi-head attention 后和/或 feed-forward layer 后。它们训练时只更新新增模块，因此比较稳定、样本效率可能较好；但推理时真的多走了一个函数模块，所以不像 LoRA 那样天然可合并到原权重中，inference efficiency 可能较差。

$$f_{\phi_i}(x) = W_U \sigma(W_D x), \quad W_D \in \mathbb{R}^{k \times d}, W_U \in \mathbb{R}^{d \times k}$$

考试问法：如果考 adapters，必须说它属于 function composition，不是直接改原权重；结构是 down-projection、activation、up-projection 的 bottleneck；常插在 attention 或 FFN 后；优点是稳定、适合多任务切换，代价是推理增加额外模块。

考试速记版

- PEFT 的动机：LLM full fine-tuning 训练显存、计算、优化器状态和多任务存储成本太高。
- 训练比推理更吃内存，因为训练要保存 gradients、activations 和 optimizer states。
- ICL 不更新参数但每次都要处理 prompt，且对 wording、example order 和随机标签现象敏感。
- PEFT 通常冻结原模型参数 θ ，只训练新增或选定模块参数 ϕ 。

- 三类 PEFT: parameter composition 改权重, input composition 改输入, function composition 加新函数模块。
- Parameter composition 包括 sparse subnetworks、structured composition 和 low-rank composition。
- BitFit 是 structured composition: 只更新 bias vectors。
- Low-rank composition 假设有效任务更新位于低维子空间, 而不是完整参数空间。
- LoRA 冻结 (W_0) , 训练 (B, A) , 用 (BA) 表示 (ΔW) 。
- LoRA 的 rank (r) 控制容量和参数量; (r) 太低可能表达不足, 太高成本上升。
- LoRA 初始化常让 $(BA=0)$, 使初始模型等于原模型, 同时训练时梯度能推动低秩参数变化。
- LoRA 常用于 Transformer self-attention 的 (W_Q, W_K, W_V) 等投影矩阵, 并可在推理时合并权重。
- Prompt tuning 学习 continuous prompt embeddings, 不一定对应人类可读文字。
- Prompt tuning 参数极少, 但占上下文、容量有限, 小模型和困难任务上表现可能较差。
- Multi-layer prompt tuning 把 prompts 加到每层, 常拼接到 self-attention keys/values。
- Adapters 属于 function composition, 典型结构是 down-projection、activation、up-projection 的 bottleneck。
- Adapters 常放在 multi-head attention 后和/或 feed-forward layer 后, 优点是稳定, 代价是推理增加额外操作。
- LoRA 和 adapters 是讲义总结中最常见的实践 PEFT 方法。
- 多任务部署中, 一个 base model 可共享, 针对不同任务切换小型 LoRA、prompt 或 adapter 参数。
- 综合比较时分开说 parameter efficiency、training efficiency、inference efficiency 和 performance。

一段稳妥考场回答

Parameter-efficient Fine-Tuning (PEFT) is motivated by the fact that full fine-tuning becomes impractical for very large language models: training must store and update all parameters plus gradients, activations and optimizer states, and separate full checkpoints for many tasks are expensive. PEFT freezes most pretrained parameters θ and trains only a small set of module parameters ϕ , aiming to approach full fine-tuning performance with much lower training and storage cost. It differs from in-context learning because ICL does not train parameters and instead puts task information in the prompt, which is inefficient at inference time and sensitive to wording and example order; PEFT requires training but stores the adaptation in small learned parameters. The lecture divides PEFT into three families. Parameter composition modifies selected weights or weight updates, including sparse subnetworks, structured choices such as BitFit, and low-rank methods. LoRA is the key example: it freezes W_0 and learns low-rank matrices B and A so that $\Delta W = BA$, often on Transformer attention projections; the rank r controls the trade-off between capacity and parameter cost, initialization can make $BA=0$ so the model initially behaves like the pretrained model, and the update can often be merged for efficient inference. Input composition instead adds learnable vectors to the input, as in prompt tuning, where continuous prompt embeddings are prepended to the input; this uses very few parameters but has limited capacity, consumes context, and can work poorly for smaller models or difficult tasks. Multi-layer prompt tuning increases capacity by adding prompts at every layer, often to attention keys and values. Function composition adds new task-specific modules, most notably adapters: bottleneck feed-forward modules with down-projection, nonlinearity and up-projection, usually inserted after attention and/or feed-forward layers. Adapters can be robust and sample-efficient but add extra computation at inference. Overall, PEFT is best understood by where the task information is stored: in weight updates for LoRA, in input vectors for prompt tuning, or in added functions for adapters.